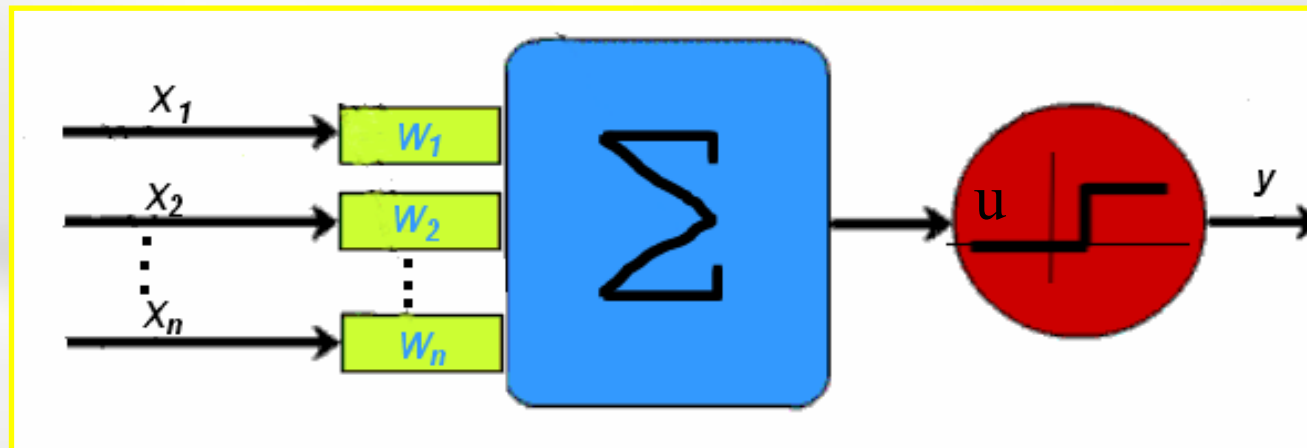


Peceptron

Model perceptronu (pierwszego neuronu)

- **Perceptron** - pierwsza formalna definicja sztucznego neuronu (McColloch i Pitts (1943))



- sygnały wejściowe $x_i \in \{0, 1\}$ dla $i = 1, 2 \dots N$
- wagi $w_i \in \{-1, 1\}$ dla $i = 1, 2 \dots N$
- wyjście neuronu

$$y = \begin{cases} 1, & \text{gdy } \sum_{i=1}^n w_i x_i \geq T \\ 0, & \text{gdy } \sum_{i=1}^n w_i x_i < T \end{cases}$$

Perceptron prosty (Frank Rosenblatt 1958)

- Perceptron zbudowany jest z neuronów ze skokową funkcją przejścia:

unipolarną

$$y = I(u) = \begin{cases} 1, & \text{dla } u > 0 \\ 0, & \text{dla } u \leq 0 \end{cases}$$

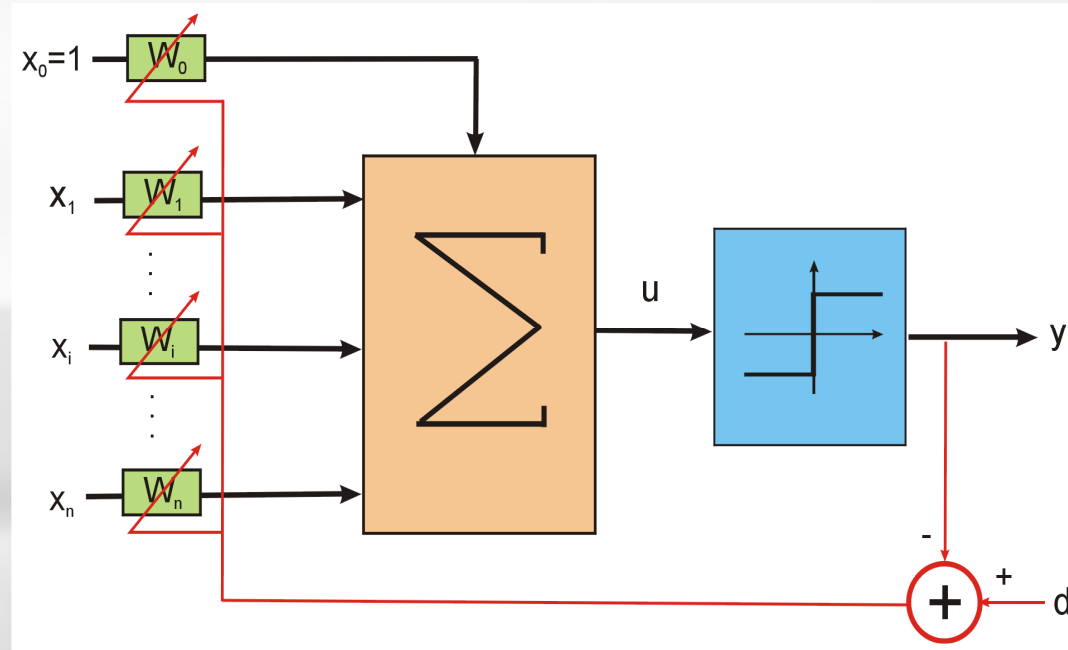
bipolarną

$$y = \text{sgn}(u) = \begin{cases} 1, & \text{dla } u > 0 \\ -1, & \text{dla } u \leq 0 \end{cases}$$

- Perceptron prosty składa się jedynie z warstwy wejściowej i warstwy wyjściowej (w praktyce każdy element jest niezależny)
- Twierdzenie Rosenblatta (1962)
Jeżeli tylko istnieje taki wektor wag w , przy pomocy którego element perceptronowy odwzoruje w sposób poprawny zbiór wzorcowych wektorów wejściowych $\{x\}$ na odpowiadający mu zbiór oczekiwanych wartości wyjściowych $\{d\}$, to istnieje metoda uczenia tego elementu gwarantująca zbieżność do wektora w

Uczenie perceptronu

- Uczenie perceptronu ma charakter uczenia z nauczycielem (nadzorowanego).



- Uczenie perceptronu podobnie jak i innych sztucznych neuronów polega na modyfikacji wektora wag:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \Delta \mathbf{w}_i(t)$$

$\mathbf{w}_i(t+1)$ – wartość wagi i-tego wejścia neuronu w chwili $t+1$ (przyszła wartość)
 $\mathbf{w}_i(t)$ – wartość wagi i-tego wejścia neuronu w chwili t (aktualna wartość),
 $\Delta \mathbf{w}_i(t)$ – zmiana wartość wag i-tego neuronu w chwili

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \Delta \mathbf{w}_i(t)$$

- Najprostszą metodą uczenia jest modyfikowanie wektora wag o ustaloną wartości proporcjonalną do popełnianego błędu

$$\Delta \mathbf{w}_i(t) = \eta \delta_\mu(t) \mathbf{x}^\mu$$

$$\delta_\mu(t) = d_\mu - y_\mu(t)$$

t – numer kroku algorytmu

η – współczynnik uczenia

d_μ – żądana odpowiedź ma μ -ty wektor wejściowy $\mathbf{x}^\mu$

y_μ – aktualna odpowiedź ma μ -ty wektor wejściowy $\mathbf{x}^\mu$

Matlab - tworzenie perceptronu

newp funkcja tworzy jednowarstwową sieć neuronową, złożoną z zadanej liczby perceptronów (sztucznych neuronów z bipolarną lub unipolarną funkcją przejścia).

net = newp(pr, s, tf, lf)

wejścia

- `pr` – macierz (o rozmiarze $R \times 2$) zawierająca min i max wartości dla R wejść sieci
- `s` – liczba neuronów w sieci;
- `tf` – funkcja aktywacji neuronów; domyślnie = 'hardlim'
- `lf` – nazwa funkcji używanej do modyfikacji wag; domyślnie = 'learnp';

wyjście

- `net` – obiekt opisujący architekturę utworzonej sieci neuronowej

Funkcją aktywacji perceptronów (parametr `tf`) oprócz funkcji unipolarnej (`hardlim`) może być funkcja unipolarna (`hardlims`)

Jako parametru `lf` możemy użyć nazw następujących funkcji uczących:

- `learnp` – modyfikacja wag w oparciu o regułę perceptronową
- `learnpn` – modyfikacja wag w oparciu o regułę perceptronową (szybsza gdy wektor wejściowy zawiera elementy o bardzo różnych wartościach)

```
net = newp(pr, s, tf, lf)
```

Przykłady:

```
net = newp([-1 1; 2.5 4], 1)
```

Podane powyżej wywołanie funkcji tworzy sieć zawierającą pojedynczy neuron o dwóch wejściach. Zakres wartości pierwszego wejścia to [-1, 1] drugiego [2.5, 4]. Wywołanie funkcji zawiera tylko dwa argumenty – pozostałe dwa przyjmą wartości domyślne: hardlim dla funkcji aktywacji neuronu oraz learnp dla funkcji korekty wag sieci.

```
net = newp([-2 1; 3 4; -2 0], 2, 'hardlims')
```

Podane powyżej wywołanie funkcji tworzy sieć zawierającą dwa neuron o trzech wejściach. Zakres wartości pierwszego wejścia to [-2, 1] drugiego [3, 4] a trzeciego [-2 0], neurony będą miały bipolarną funkcję przejścia (hardlims).

Matlab - uczenie perceptronu (`train`)

`train`

Funkcja realizuje trening sieci neuronowej, wykorzystując funkcję treningu, której nazwa została podana w polu `net.trainFcn`, zaś wartości niezbędnych parametrów – w polu `net.trainParam`.

Funkcja treningu jest uniwersalna służy do uczenia wszystkich typów sieci

```
[net, tr, Y, E, Pf, Af] =  
    train(net, P, T, Pi, Ai, VV, TV)
```

Aktualnie ograniczymy się do najprostszego jej wywołania

```
net = train(net, P, T)
```



```
net = train(net,P,T)
```

Wejścia

- `net` - struktura (obiekt) zawierająca opis wielowarstwowej sieci jednokierunkowej
- `P` - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy),
- `T` - macierz pożądaných odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi), dla sieci uczonych bez nauczyciela ten argument przyjmuje domyślnie wartość zerową,

Wyjście

- `net` - struktura (obiekt) zawierająca opis nauczonej sieci jednokierunkowej

Matlab - parametry uczenia `net.trainParam`

Obiekt `net` zawiera grupę pól `trainParam` w których określa się parametry uczenia sieci – oto wybrane z nich:

- `net.trainParam.epochs` maksymalna ilość epok uczenia (domyślnie 100)
- `net.trainParam.goal` maks. dopuszczalna wartość funkcji celu – najczęściej jest nią suma kwadratów błędu (domyślnie 0)
- `net.trainParam.mu` współczynnik uczenia domyślnie 0.001
- `net.trainParam.show` określa co ile epok funkcja ma informować o postępach uczenia (domyślnie 25)

Matlab - uczenie perceptronu (adapt)

adapt

Funkcja realizuje jeden krok (tzw. epokę) treningu sieci neuronowej, dokonując adaptacji wag sieci. Do adaptacji jest wykorzystywana funkcja podana w polu `net.adaptFcn`, zaś wartości niezbędnych parametrów tej funkcji – w polu `net.adaptParam`.

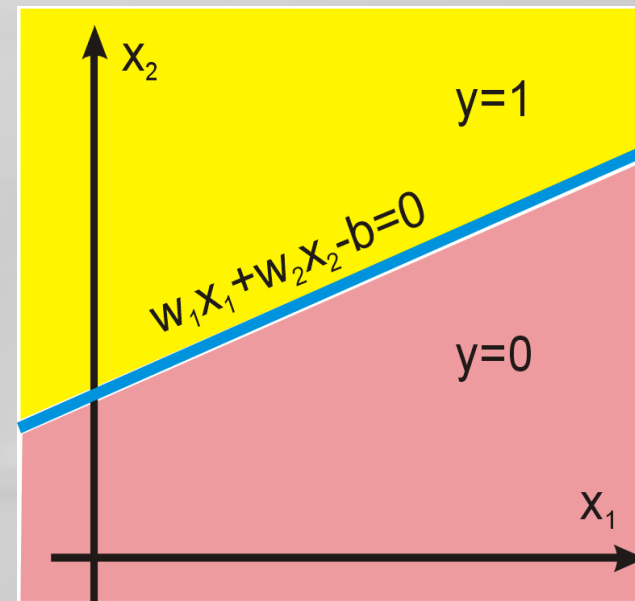
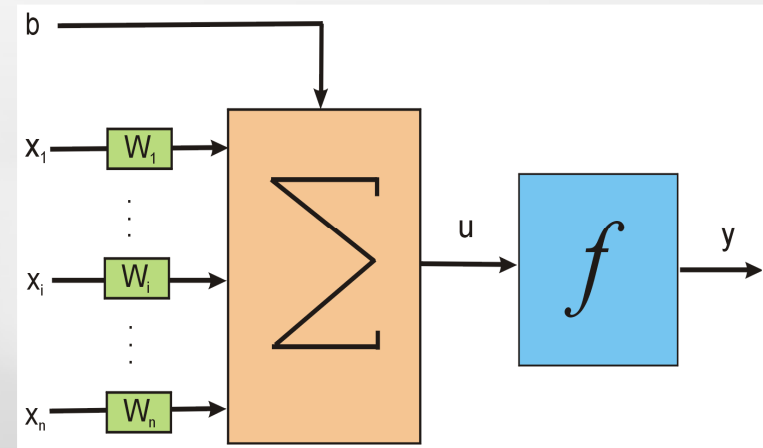
Funkcja ma taką samą postać dla wszystkich sieci.

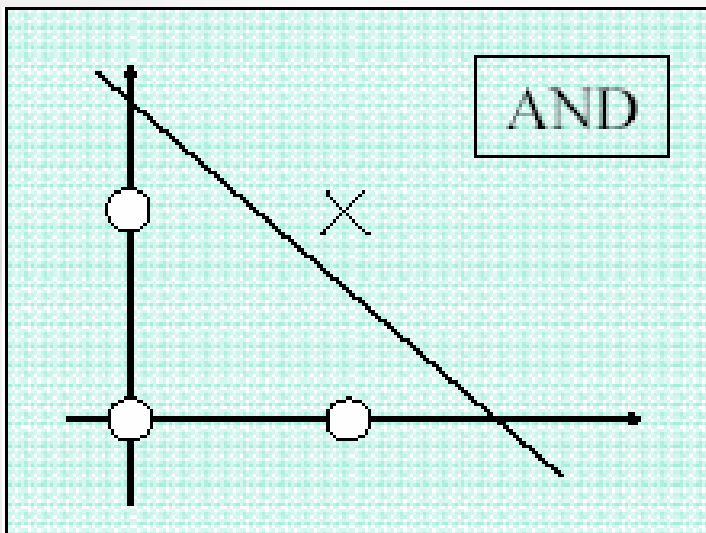
```
[net,y,e,Pf,Af,tr] = adapt(net,P,T,Pi,Ai)
```

Parametry funkcji `adapt` są identyczne jak funkcji `train`

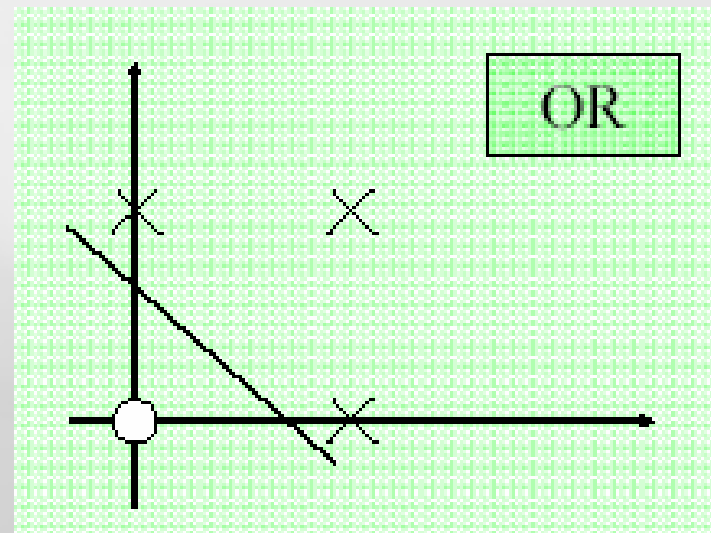
Co potrafi perceptron?

- Rozważmy perceptron o n wejściach i progu aktywacji b .
- Element taki dzieli $(n-1)$ wymiarową przestrzeń na dwie półprzestrzenie – w przypadku dwuwymiarowym Równanie perceptronu można traktować jako równanie prostej
- Punkty leżące nad ową prostą klasyfikowane są jako 1, zaś pozostałe jako 0.





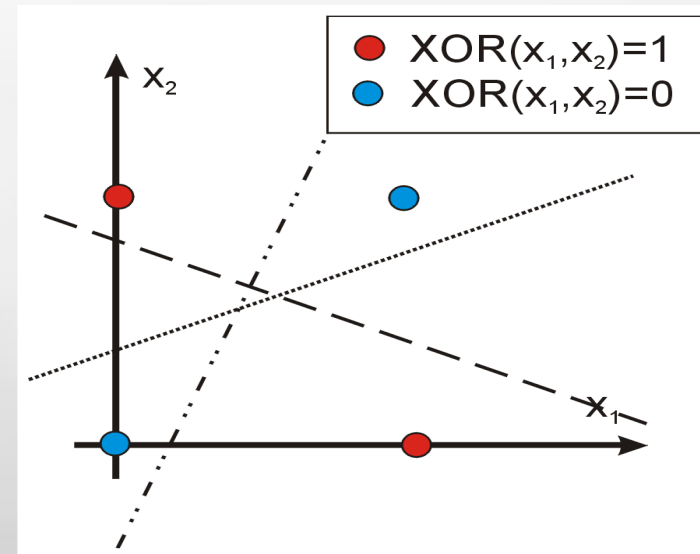
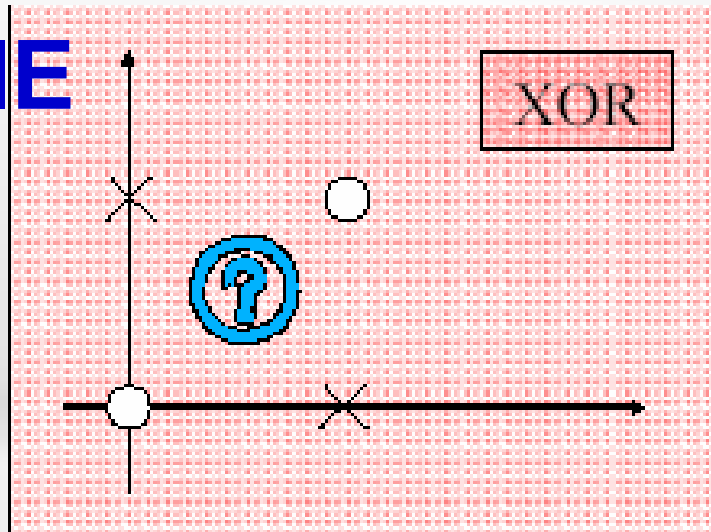
TAK



- Perceptron potrafi rozdzielić zbiory liniowo separowalne

Czego perceptron nie potrafi

NIE



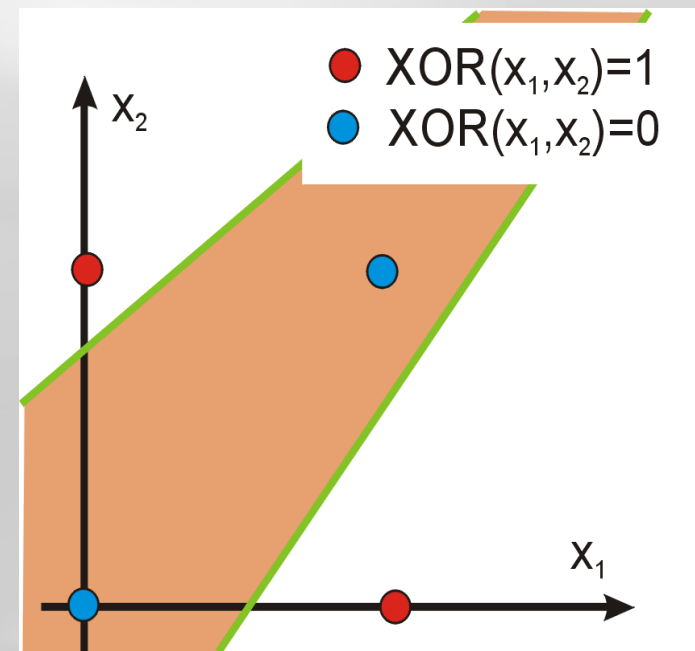
- Pojedynczy perceptron nie potrafi odróżniać zbiorów **nieseparowalnych liniowo**, np. funkcji XOR – zauważyli to Minsky i Papert (1969)
- Odkrycie ograniczeń perceptronu na wiele lat zahamowało rozwój sieci neuronowych.

Perceptron prosty - wnioski

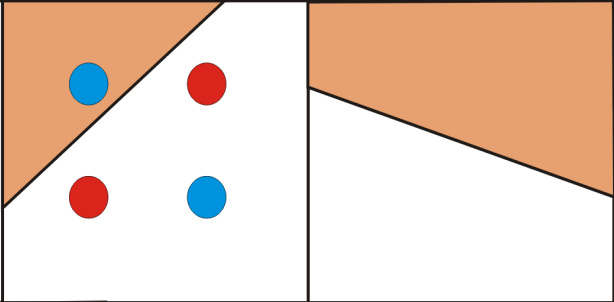
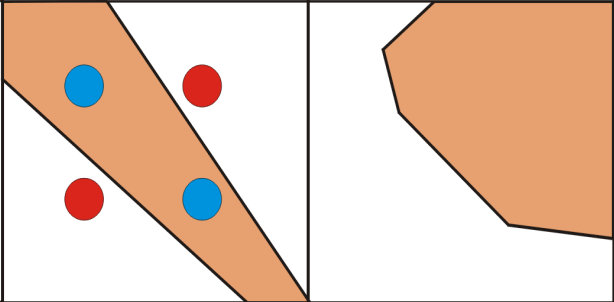
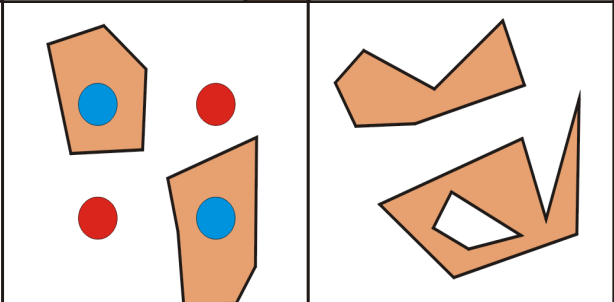
- Zadaniem pojedynczego perceptronu jest jedynie:
 - przetwarzanie jednostkowych informacji
 - podejmowanie prostych decyzji
 - przekazywanie wyników sąsiadom
- Dopiero w połączeniu z innymi węzłami uzyskuje się zdolność podejmowania złożonych decyzji

Perceptron wielowarstwowy

- Jeżeli połączymy perceptrony proste w układ warstwowy to usuniemy ograniczenie liniowej separowalności
- Perceptron dwuwarstwowy potrafi rozwiązać problem XOR
- Perceptron trójwarstwowy potrafi zrealizować dowolne odwzorowanie w przestrzeni stanów.



Zestawienie możliwości perceptronów wielowarstwowych

pojedyncza warstwa	
sieć dwuwarstwowa	
sieć trójwarstwowa	

Przykładowe zadanie

Napisać skrypt w matlabie, który stworzy i wytrenuje (o ile to możliwe) sieć neuronową złożoną z dwóch neuronów (perceptronów prostych) klasyfikacji następujących elementów:

$a_1=(2,11)$, $a_2=(7,10)$, $a_3=(8,16)$,

$b_1=(10,6)$, $b_2=(15,10)$,

$c_1=(-4,6)$, $c_2=(0,7)$,

$d_1=(-6,-7)$, $d_2=(-10,-4)$, $d_3=(-5,-15)$

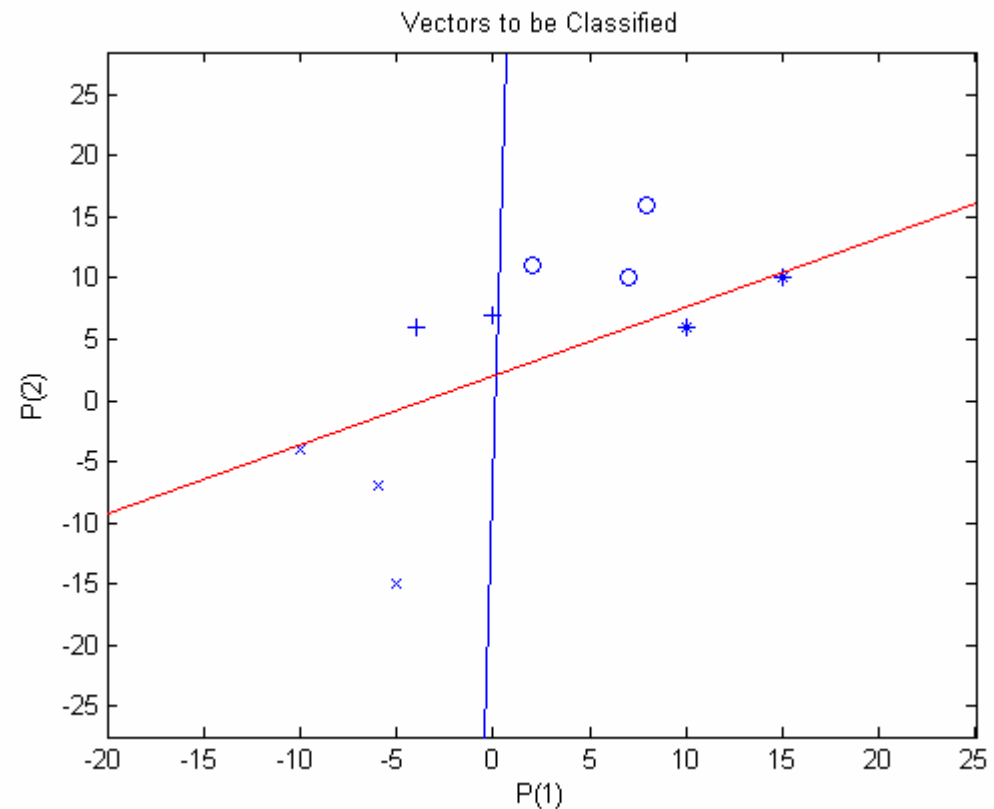
do czterech zbiorów A, B, C, D:

$A=\{a_1,a_2,a_3\}$,

$B=\{b_1,b_2\}$,

$C=\{c_1,c_2\}$,

$D=\{d_1,d_2,d_3\}$



Rozwiązanie wykorzystujące funkcje train

```
p=[2  7  8  10 15  -4 0 -5  -6 -10
    11 10 16 6  10  6 7  -15 -7 -4];
t=[0 0 0 0 0 1 1 1 1 1;
    0 0 0 1 1 0 0 1 1 1]
net=newp(minmax(p),2);
[net,tr,Y]=train(net,p,t);
figure
plotpv(p,t);
plotpc(net.IW{1,1},net.b{1});
```

Definicja wektorów wejściowych

$a_1=(2,11)$, $a_2=(7,10)$, $a_3=(8,16)$,
 $b_1=(10,6)$, $b_2=(15,10)$,
 $c_1=(-4,6)$, $c_2=(0,7)$,
 $d_1=(-6,-7)$, $d_2=(-10,-4)$, $d_3=(-5,-15)$

	a_1	a_2	a_3	b_1	b_2	c_1	c_2	d_3	d_1	d_2
$p=[$	2	7	8	10	15	-4	0	-5	-6	-10
	11	10	16	6	10	6	7	-15	-7	-4]

Definicja oczekiwanych odpowiedzi

Potrzebujemy wyróżnić 4 zbiory (A, B, C, D) i wiemy, że sieć składa się z dwóch neuronów, możemy więc zastosowywać kod 0 0 – zbiór A, 0 1 – B, 1 0 – C, 1 1 – D.

Tym sposobem możemy otrzymać oczekiwane odpowiedzi sieci:

	A	A	A	B	B	C	C	D	D	D
t = [	0	0	0	0	0	1	1	1	1	1
	0	0	0	1	1	0	0	1	1	1]

	a1	a2	a3	b1	b2	c1	c2	d3	d1	d2
p = [	2	7	8	10	15	-4	0	-5	-6	-10
	11	10	16	6	10	6	7	-15	-7	-4]

Utworzenie sieci

Sieć zawiera 1 warstwę złożoną z dwóch neuronów. Dzięki użyciu funkcji **minmax** nie musimy wpisywać bezpośrednio ekstremalnych wartości wejść sieci.

```
net=newp(minmax(p),2);
```

Trening sieci

Dodatkowo oprócz obiektu net (sieci) zwracamy odpowiedzi wytrenowanej sieci (Y) oraz przebieg procesu uczenia (tr)

```
[net,tr,Y]=train(net,p,t);
```

Wyrysowanie elementów do klasyfikacji

```
plotpv(p,t);
```

Wyrysowanie prostych separujących zbiory

```
plotpc(net.IW{1,1},net.b{1});
```

Rozwiązanie wykorzystujące funkcje adapt

```
p=[ 2   7   8 10  15  -4   0   -5  -6 -10
    11 10 16   6  10   6   7  -15  -7  -4];
t=[0 0 0 0 0 1 1 1 1 1
    0 0 0 1 1 0 0 1 1 1];
net=newp([-10 15;-15 16],2);
plotpv(p,t);
E=1;
i=0;
line=plotpc(net.IW{1},net.b{1});
while(sse(E))
    [net,Y,E]=adapt(net,p,t);
    line=plotpc(net.IW{1,1},net.b{1},line);
    drawnow;
    i=i+1;
end;
Y
E
i
```

Funkcja `plotpv` - obraz zadania dla perceptronu

`plotpv`

funkcja wykreśla położenie dla perceptronu punktów oczekiwanych odpowiedzi w funkcji wektorów wejściowych

`plotpv(P, T)`

Wejścia

- P - macierz wektorów wejściowych (każda kolumna macierzy to jeden wektor wejściowy),
- T - macierz pożądaných odpowiedzi sieci (każda kolumna macierzy to jeden wektor oczekiwanych odpowiedzi)

Funkcja `plotpc` – granica decyzyjna perceptronu

`plotpc`

Funkcja wykreśla granicę decyzyjną w przestrzeni wag sieci neuronowej, złożonej z perceptronów. Wywołanie funkcji ma sens, gdy wymiarowość wektora wejściowego sieci jest nie większa niż trzy.

```
line = plotpc (W, b, line)
```

Wejścia

- `W` – macierz wag synaptycznych perceptronu (o wymiarach $S \times R$, gdzie S jest liczbą neuronów w warstwie, zaś R – wymiarowością wektora wejść sieci, $R > 3$)
- `b` – wektor wartości progów perceptronu (o wymiarach $S \times 1$)
- `line` – nazwa ostatnio narysowanej linii (opcjonalne)

Wyjścia

- `line` – nazwa rysowanej linii (opcjonalne). W przypadku wykorzystania funkcji w „pętli”, dzięki użyciu tego parametru kolejne wywołania funkcji usuwają poprzednio wyrysowaną linię.

Instrukcje poniżej definiują wejścia i wyjścia perceptronu i ukazują je na płaszczyźnie a następnie po zainicjowaniu sieci wykreślają granice decyzyjną

```
p = [0 0 1 1; 0 1 0 1];    % macierz wektorów wejściowych
t = [0 1 1 1];             % macierz oczekiwanych odpowiedzi
plotpv(p, t)               % wykreslenie punktów na płaszz.
net = newp(minmax(p), 1);   % inicjacja sieci
net.iw{1,1} = [-1.2 -0.5]; % określenie wag
net.b{1} = 1;              % określenie biasu
plotpc(net.iw{1,1}, net.b{1}) % wykreslenie granicy decyzyjnej
```

